

Verifikacija

Lazar Dačković

Mart 2026

1 Uvod

Verifikacija softvera predstavlja ključni deo razvoja aplikacija, osiguravajući njihovu ispravnost, efikasnost i sigurnost. U okviru ovog projekta analiziran je kod programa *File Manager* koji je razvijen u C++ jeziku uz korišćenje Qt frameworka. Aplikacija sadrži dva potprozora koja listaju sadržaje direktorijuma. Nad tim fajlovima moguće je raditi razne akcije, kao što su copy/paste, cut, rename, delete. Može se pristupiti nekim dodatnim podešavanjima, menjati prava pristupa. Moguće je kreirati nove foldere, kretati se kroz foldere, premeštati fajlove iz jednog potprozora u drugi prevlačenjem.

U cilju provere ispravnosti i optimizacije koda, korišćeni su alati kao što su Cppcheck, Clnag-Tidy, Valgrind, CLazy, Heaptrack. Pomoću ovih alata analizirana je statička i dinamička struktura koda, otkrivene su potencijalne greške, analizirana memorijska potrošnja. Korišćenjem statičke analize omogućeno je rano otkrivanje logičkih grešaka, dok je dinamičkom analizom procenjena efikasnost izvršavanja programa i potencijalni problemi sa memorijom.

Pre pokretanja alata potrebno je izgraditi projekat. Pokretanjem **build.sh** skripte briše se postojeći build direktorijum (ukoliko postoji) i kreira novi. Zatim se pomoću **qmake** alata generišu build fajlovi za **debug** režim. Nakon toga se projekat kompajlira korišćenjem alata **make**, pri čemu alat **Bear** generiše **compile.commands.json** fajl neophodan za rad Clang-Tidy alata.

2 Clang-Tidy

Clang-Tidy je alat za statičku analizu koda koji je sastavni deo Clang ekosistema i namenjen je unapredjenju kvaliteta C i C++ koda. Njegova osnovna svrha je otkrivanje grešaka, analiza strukture koda i pimenjena preporučenih praksi za poboljšanje čitljivosti i optimizacije.

Alat omogućava prepoznavanje sintaksičkih i logičkih grešaka pre faze kompajliranja, pomažući u ranom otkrivanju potencijalnih problema. Takođe, Clang-Tidy nudi podršku za automatizovano formatiranje koda prema definisanim stilskim pravilima, osiguravajući doslednost u kodnoj bazi.

Još jedna značajna funkcionalnost je mogućnost automatskog ispravljanja određenih tipova problema, kao što su optimizacija izraza, dodavanje nedostajućih

zaglavlja i poboljšanje kompatibilnosti sa modernim standardima jezika. Clang-Tidy se može koristiti iz komandne linije ili integrisati u razvojna okruženja, što ga čini praktičnim alatom za kontinuiranu analizu koda i održavanje visokih standarda programiranja.

2.1 Pokretanje Clang-Tidy alata

Alat Clang-Tidy pokrenut je iz komandne linije nakon pozicioniranja u direktorijum skripte. Clang-Tidy se pokreće izvršavanjem skripte **clang_tidy_script.sh**, koja omogućava statičku analizu C++ koda sa fokusom na modernizaciju, bezbednost, stil i potencijalne greške u implementaciji.

Analiza koristi skup provera iz kategorija

- clang-diagnostic-*: greške koje bi kompajler javio
- clang-analyzer-*: napredna statička analiza
- modernize-*
- performance-*
- readability-*
- bugprone-*: sumnjivi i rizični delovi koda

i rezultat se čuva u fajlu **MyCTReport.txt**. I proveravaju se samo fajlovi iz foldera sources i headers, odnosno samo oni fajlovi koje smo sami pisali, ne Qtove ili neke druge biblioteke.

2.2 Otkriveni problemi i preporučene optimizacije

U nastavku su predstavljeni najznačajniji nalazi Clang-Tidy analize, zajedno sa preporučenim rešenjima.

2.2.1 Korišćenje ključne reči auto

Auto je ključna reč koja omogućava da kompajler automatski zaključi tip promenljive, i poboljšava čitljivosti koda.

U kodu: `QFile* file = new QFile(ui->leUploadFileInput->text());`

Preporučeno: `auto file = new QFile(ui->leUploadFileInput->text());`

2.2.2 Zamena `size()>0` sa `!empty()`

Čitljivije i funkcija odmah vraća bool vrednost, `.size()` može biti neefikasno za neke tipove jer mora da računa veličinu

U kodu: `if (m_fileList.size() > 0)`

Preporučeno: `if (!m_fileList.empty())`

2.2.3 Korišćenje range-based for petlji

Čitljiviji kod, ne dolazimo u opasnost od pristupanja elementu van opsega, nema nepotrebnog kopiranja.

```
U kodu: for (int j = 0; j < m_fileList.size(); j++){
    ...
    file->write(QString(m_fileList.at(j) + ",").toUtf8());
    ...
}
Preporučeno: for(const auto &item : m_fileList){
    ...
    file->write(QString(item + ",").toUtf8());
    ...
}
```

2.2.4 Prosledjivanje argumenata po referenci

Izbegava se nepotrebno kopiranje.

U kodu:

```
void Preview::processFile(const File file, QMap<QString, int> &extMap)
```

Preporučeno:

```
void Preview::processFile(const File &file, QMap<QString, int> &extMap)
```

3 Cppcheck

Cppcheck je moćan alat za analizu koda namenjen isključivo C i C++ programima, sa ciljem otkrivanja različitih vrsta grešaka, problema sa memorijom i potencijalnih bezbednosnih propusta. Njegova ključna prednost je mogućnost analize bez potrebe za kompajliranjem koda, čime omogućava rano prepoznavanje problema pre izvršavanja programa.

Alat identifikuje sintaksičke i logičke greške, neoptimalne konstrukcije i nedostatke u upravljanju resursima, što olakšava poboljšanje efikasnosti i stabilnosti aplikacija. Može se koristiti kroz komandu liniju, ali je takodje kompatibilan sa različitim razvojnim okruženjima. Cppcheck pruža fleksibilne opcije izveštavanja, omogućavajući programerima dublju analizu koda i otkrivanje skrivenih problema.

3.1 Analiza

Cppcheck se pokreće izvršavanjem skripte **cppcheck.sh**, koja omogućava statičku analizu C++ koda radi otkrivanja potencijalnih grešaka, problema sa memorijom i sigurnošću programa.

3.1.1 Problemi sa `cstyleCast`

U kodu se koristi C-style cast `((Type*))`, što u C++ nije preporučeno. Nebezbedno je i može sakriti bagove.

U kodu: `auto mainWindow = (MainWindow*)m_parent->getParent();`

Preporučeno: `auto mainWindow = qobject_cast<MainWindow*>(m_parent->getParent());`

3.1.2 Deklaracija promenljive izvan bloka korišćenja i postfiks operator

Promenljive imaju širi obim nego što je to potrebno. Smanjuje čitljivost koda i povećava šansu za greške. Postfiks operator pravi privremenu kopiju, za primitivne tipove to i nije neki problem, ali za objekte i iteratore pravi nepotrebnu kopiju.

U kodu:

```
int m; int n;
 QMap <QString,int> ::iterator i;
 for(i=extMap2.begin(); i!=extMap2.end();i++){
 ...
     m=i.value();
 ...
 }
```

Preporučeno: *Uočio sam i da je moguće koristiti ključnu reč `auto` u `for` petlji*

```
for(auto i=extMap2.begin(); i!=extMap2.end();++i){
 ...
     int m=i.value();
 ...
 }
```

4 Valgrind

Valgrind je moćan alat za analizu i profilisanje softvera, prvenstveno korišćen u C i C++ programiranju, ali može biti primenjen i na druge jezike. Njegova glavna svrha je detekcija grešaka u memoriji, optimizacija performansi i analiza konkurentnog izvršavanja. Funkcioniše tako što pokreće program unutar sopstvenog virtuelnog okruženja, omogućavajući detaljno praćenje svih memorijskih operacija i izvršavanje funkcija.

Ovaj alat se često koristi u razvoju softvera gde je potrebno osigurati stabilnost, sprečiti curenje memorije i otkriti potencijalne probleme u upravljanju resursima. Valgrind omogućava programerima da testiraju svoje aplikacije bez potrebe za dodatnim modifikacijama koda, pružajući detaljne izveštaje o eventualnim greškama koje mogu dovesti do nepredvidivog ponašanja programa.

Glavni alati u okviru Valgrind paketa su: **Memcheck**, **Cachegrind**, **Callgrind**, **Massif**, **Helgrind** i **DRD**. U ovom projektu je korišćen alat Memcheck, kako bi se proverila ispravnost upravljanja memorijom u aplikaciji. Ovaj alat

detektuje curenje memorije, nevalidne pristupe i druge potencijalne probleme u dinamičkoj alokaciji memorije.

4.1 Analiza

Valgrind se pokreće izvršavanjem skripte **valgrind.sh**, i to je dinamički alat i rezultat zavisi od konkretnog načina izvršavanja. Izveštaj se nalazi u fajlu **valgrind_report.txt**. Pokrenut je sa opcijom **-leak-check=full**, koja nam daje detaljnu analizu curenja memorije, tačan stack trace i liniju koda. I sa opcijom **show-leak-kinds=definite** koji nam pokazuje sigurna curenja.

4.1.1 Curenje memorije, objekat je kreiran sa new, a nema delete

U kodu:

```
void MainWindow::toggle_darkMode(){
QFile *styleFile = m_ui->action_darkMode->isChecked()
? new QFile(":/stylesheets/darkTheme.qss")
: new QFile(":/stylesheets/lightTheme.qss");

styleFile->open( QFile::ReadOnly );
QString style( styleFile->readAll() );
this->setStyleSheet( style );
}
```

Preporuka: Na kraju funkcije treba samo dodati (**delete styleFile;**). Ili ne kreirati objekat na hipu sa **new**, već na steku, tada nam nije potreban delete.

4.1.2 Više problema sa klasom Clipboard

U klasi Clipboard identifikovani su sledeći problemi:

Klasa nema destruktor koji bi oslobodio memoriju alociranu u konstruktoru (**m_list = new SelectedFilesList()**), što dovodi do curenja memorije.

Preporučuje se dodavanje destruktora koji oslobadja **m_list**

U funkciji **set()** kreira se novi objekat pomoću **new** bez prethodnog brisanja postojećeg (**delete**), što takodje uzrokuje curenje memorije.

U kodu:

```
void Clipboard::set(const SelectedFilesList *list){
    m_list->clear();
    m_list = new SelectedFilesList(*list);
}
```

Preporuka:

```
void Clipboard::set(const SelectedFilesList *list){
    delete m_list;
    m_list = new SelectedFilesList(*list);
}
```

5 Clazy

Clazy je statički analizator razvijen kao dodatak za Clang. Clazy je specijalizovan za analizu C++ koda koji koristi Qt biblioteke i omogućava detekciju problema koji nisu obuhvaćeni standardnim kompajlerskim upozorenjima. Njegov cilj je da unapredi kvalitet koda kroz identifikaciju loših praksi, potencijalnih grešaka i neefikasnih operacija karakterističnih za Qt.

5.1 Analiza

Alat Clazy se integriše u proces kompajliranja, posmatra kod dok se prevodi. Pokrećemo ga pomoću skripte **CLazy.sh** koja će nam dati dva izveštaja, jedan potpun nazvan `temp_report.txt` i jedan sažet koji se zove `clazy_report.txt` koji se odnosi samo na fajlove koji su pisani od strane programera koji su radili na aplikaciji File Manager.

5.1.1 Zastarela funkcija `created()`

Koristi se zastarela funkcija `created()` koja dolazi iz klase `QFileInfo`. Treba je zameniti radi bolje kompatibilnosti i uskladjenosti sa novijim verzijama Qt biblioteke.

U kodu: `m_created(fileInfo.created())`

Preporučeno: `m_created(fileInfo.birthTime())`

5.1.2 Prosledjivanje po vrednosti netrivialnih tipova

Argumenti u konstruktoru klase `File` se prosledjuju po vrednosti, i prave se nepotrebne kopije što može biti neefikasno ako se konstruktor često poziva.

Preporuka:

```
File::File(const QString &name, const QString &path, qint64 size,
          const QString &extension,
          const QDateTime &created,
          const QDateTime &modified,
          const QFileDevice::Permissions permissions,
          bool isDir,
          const QDateTime &opened)
```

5.1.3 Korišćenje range-based for petlji

Bolje je koristiti referencu kako bi se izbeglo kopiranje svakog `QString`-a.

Preporuka:

```
for(const QString &filePath : dir.entryList(fileFilters))
```

5.1.4 `QString` literal warning

Clazy preporučuje da npr `QString s = "text";` ne konstruišemo tako, već `QString s = QStringLiteral("text");`

6 Heaptrack

Heaptrack je napredan alat za profilisanje dinamičke memorije na Linux sistemima. Njegova osnovna svrha je praćenje svih alokacija i dealokacija memorije tokom izvršavanja programa. Za razliku od nekih drugih alata, Heaptrack je optimizovan tako da ima relativno mali uticaj na performanse aplikacije, što ga čini pogodnim za analizu i većih i kompleksnijih sistema.

Rezultati koje generiše Heaptrack analiziraju se pomoću grafičkog interfejsa `heaptrack_gui`, koji omogućava vizuelno istraživanje potrošnje memorije. Ovaj alat pruža detaljan uvid u to gde i kako program koristi memoriju, uključujući informacije o broju alokacija, količini zauzete memorije, kao i potencijalnim mestima curenja memorije.

6.1 Analiza

Heaptrack se pokreće izvršavanjem skripte **heaptrack.sh**, to je takodje dinamički alat i rezultat zavisi od konkretnog načina izvršavanja. Izveštaj se nalazi u fajlu **heaptrack.FileManager.13253.zst**, koji može biti pokrenut uz pomoć alata `heaptrack_gui` radi bolje preglednosti. Program je korišćen na način da se provere funkcionalnosti View, Preview, DarkMode itd.

6.1.1 Curenje memorije

Pri svakom kliku na dugme View kreira se objekat `QChartView` na hipu koji nam prikazuje grafikone, ali se nikada ne briše iz memorije. Klasično rešenje sa `delete` na kraju funkcije ovde neće raditi, jer bi to značilo da će grafikon istog momenta da se nacrti i obriše.

Preporučeno: Dodati ovaj atribut da se samouništi kada više nije potreban.
`chartView->setAttribute(Qt::WA_DeleteOnClose);`

6.1.2 Prevelik PEAK

Nemamo konkretno curenje memorije, ali imamo trenutak kada nepotrebno zauzimamo resurse. Prilikom preview slike, učitavamo celu sliku u memoriju, da bismo je prikazali u malom prozorčiću u preview modu.

U kodu:

```
QPixmap p(file.getPath()); // <--- OVDE JE PROBLEM
m_w=m_graphicsView->width();
m_h=m_graphicsView->height();
m_graphicsView->scene()->addPixmap(p.scaled(m_w, m_h, Qt::KeepAspectRatio));
```

Preporuka:

```
QPixmap p = QPixmap::fromImage(QImage(file.getPath()).scaled(m_w, m_h,
Qt::KeepAspectRatio, Qt::SmoothTransformation));
```

7 Flawfinder

Flawfinder je statički alat za analizu koda namenjen isključivo C i C++ programima, sa ciljem otkrivanja potencijalnih bezbednosnih propusta i ranjivosti.

Alat identifikuje poznate bezbednosne propuste, kao što su problemi sa prekoračenjem bafera (buffer overflow) i nebezbedne funkcije, što olakšava poboljšanje otpornosti i sigurnosti aplikacija. Može se koristiti kroz komandnu liniju, ali je takodje kompatibilan sa različitim razvojnim okruženjima. Flawfinder pruža fleksibilne opcije izveštavanja i rangiranja rizika po nivoima ozbiljnosti, omogućavajući programerima dublju analizu koda i otkrivanje skrivenih propusta.

7.1 Analiza

Flawfinder se pokreće izvršavanjem skripte **flawfinder.sh** i rezultat se čuva u **flaw_html.html** fajlu. Pokrenut je nad direktorijumima `sources` i `headers` gde se nalaze fajlovi pisani od strane programera koji su razvijali projekat i ispisuje upozorenja koje su 3. ili višeg nivoa. U kodu nisu pronadjene problematične funkcije.